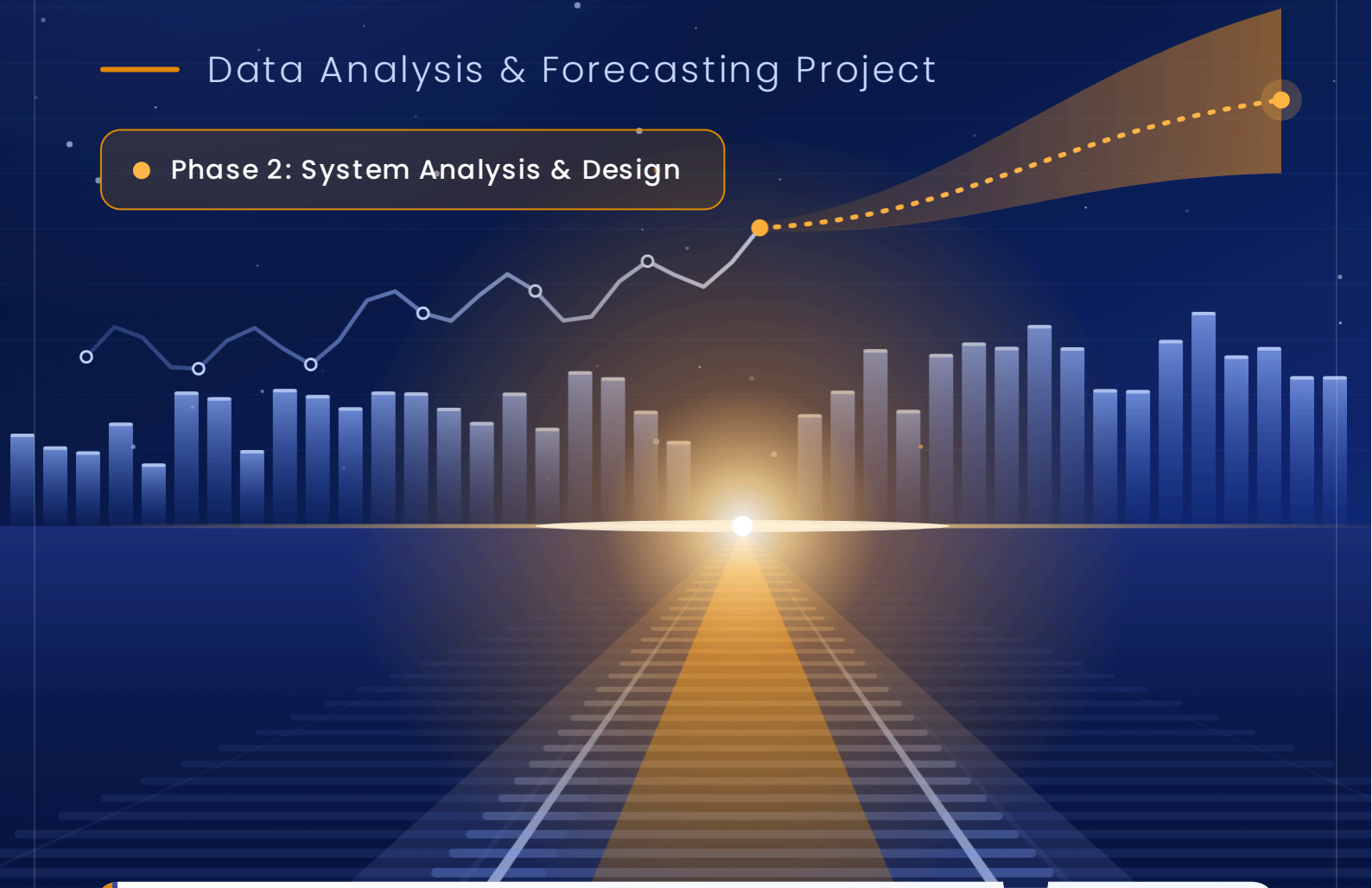
DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

UK Train Rides

Data Analysis & Forecasting Project

Phase 2: System Analysis & Design



TEAM MEMBERS

1. Adham Reda
2. Ahmed Nasser
3. Mohamed Emam
4. Mohamed Ibrahim
5. Yousef Mohamed
6. Yousef Sherif

Requirements Map

SOURCES: *Team 6 Phase 2 brief (11 requirements); DEPI "Project Documentation Guidelines", Section 4 - System Analysis & Design (Industry DEPI deadline: 6 Nov 2026).*

The table lists the 11 requirements of Phase 2. Items 1-8 are the diagram and design deliverables named in DEPI Section 4; items 9-11 are the additional items requested by the team. Every section in this document opens with the mandatory banner stating its requirement number and official name.

No.	Official requirement	Description
1	Use Case Diagram	2 actors (Decision Maker, Data Analyst) and 6 use cases of the UK Train Rides system, with Mermaid code and descriptions. [DEPI Sec. 4]
2	DFD Level 0 & Level 1	Context-level flow (User, system, Dashboard) and the 5 detailed processes from railway.csv ingestion to Tableau. [DEPI Sec. 4]
3	Sequence Diagram	Scenario "Request May 2024 Forecast": Decision Maker, Dashboard, SQL, ForecastModel. [DEPI Sec. 4]
4	Activity Diagram	End-to-end workflow from CSV upload and cleaning to MAPE evaluation and dashboard build. [DEPI Sec. 4]
5	State Diagram	Life cycle of a journey: Scheduled, On Time, Delayed, Cancelled, Refund Requested, Refund Approved. [DEPI Sec. 4]
6	Class Diagram	Ticket, Journey, Station and ForecastModel classes with attributes, methods and relationships. [DEPI Sec. 4]
7	ER Diagram + Logical Schema + Physical Schema (SQL)	Stations / Tickets / Journeys model, logical tables and executable SQL Server (T-SQL) script with PK, FK, UNIQUE and CHECK. [DEPI Sec. 4]
8	Wireframes + Technology Stack	7 Tableau dashboard pages (KPI cards on every page) with page-by-page analysis, and the Layer / Tool / Justification table. [DEPI Sec. 4]
9	Data Dictionary	All 18 columns of railway.csv with type, example, description and known data issues. [Team brief - additional]
10	Assumptions & Constraints	Assumptions and constraints that bound the analysis and the May 2024 forecasts. [Team brief - additional]
11	Non-Functional Requirements	Performance, usability, accuracy and security targets. [Team brief - additional; DEPI Sec. 3-4 list NFRs]

Beyond the 11 items, DEPI Section 4 also lists Problem Statement & Objectives, Functional Requirements, Software Architecture, Component and Deployment diagrams and UI/UX Guidelines. Appendix A covers all of these except the Deployment Diagram, which is not included because the project brief does not define a hosting environment.

1 Use Case Diagram

SOURCES: *Team brief (actors, 6 use cases); DEPI Sec. 4 "Use Case Diagram & Descriptions"; project goals 1-4 from the brief.*

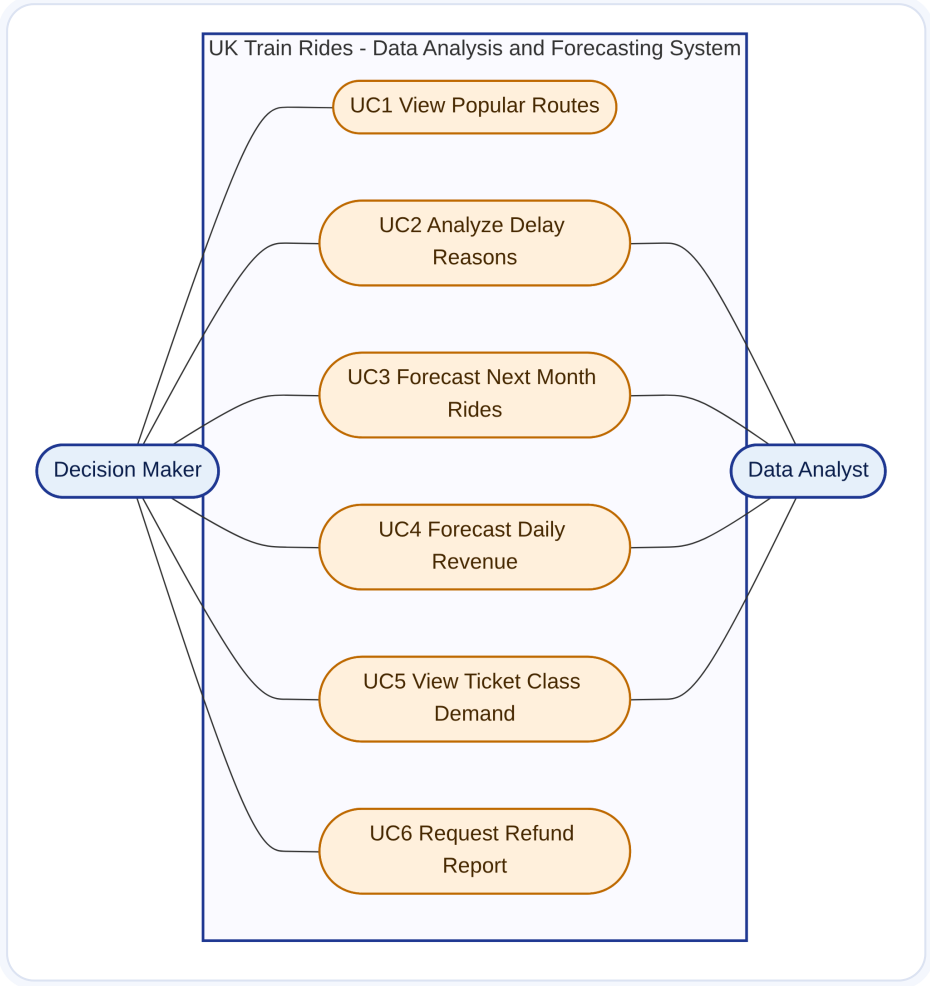


Figure 1 *- Use case diagram of the UK Train Rides analysis system*

Mermaid code

```
flowchart LR
  DM(["Decision Maker"])
  DA(["Data Analyst"])
  subgraph SYS["UK Train Rides - Data Analysis and Forecasting System"]
    UC1(["UC1 View Popular Routes"])
    UC2(["UC2 Analyze Delay Reasons"])
    UC3(["UC3 Forecast Next Month Rides"])
    UC4(["UC4 Forecast Daily Revenue"])
    UC5(["UC5 View Ticket Class Demand"])
    UC6(["UC6 Request Refund Report"])
  end
  DM --- UC1
  DM --- UC2
  DM --- UC3
  DM --- UC4
  DM --- UC5
  DM --- UC6
  UC2 --- DA
  UC3 --- DA
  UC4 --- DA
  UC5 --- DA
  classDef actor fill:#E8F0FE,stroke:#1F3A93,stroke-width:2px,color:#0B1F4D
  classDef uc fill:#FFF4E0,stroke:#C26A00,stroke-width:1.5px,color:#4A2800
  class DM,DA actor
  class UC1,UC2,UC3,UC4,UC5,UC6 uc
  style SYS fill:#FAFBFF,stroke:#1F3A93,stroke-width:2px
```

Actors

Actor	Description	Use cases
Decision Maker	Views the reports and forecasts (routes, delays, refunds, May 2024 demand) to support operational and commercial decisions.	UC1, UC2, UC3, UC4, UC5, UC6
Data Analyst	Cleans the data, runs the forecasts and builds the Tableau dashboard that serves the Decision Maker.	UC2, UC3, UC4, UC5 (produces the analysis behind UC1 and UC6 as well)

Use cases

ID	Use case	Description	Project goal
UC1	View Popular Routes	Top 10 routes by rides and by revenue; London Euston - Birmingham New Street is the 2nd busiest route (4,209 rides).	Goal 1: profit and routes
UC2	Analyze Delay Reasons	Distribution of delay reasons after the 8 labels are merged into 5 causes; Weather is the leading cause (1,372 of the 4,172 delayed or cancelled journeys).	Goal 2: delay and refund
UC3	Forecast Next Month Rides	Forecast of the number of rides for May 2024.	Goal 3: May 2024 forecast
UC4	Forecast Daily Revenue	Forecast of the average daily revenue for May 2024.	Goal 3: May 2024 forecast
UC5	View Ticket Class Demand	Forecast of demand per ticket class (Standard vs First Class) for May 2024.	Goal 4: ticket class demand
UC6	Request Refund Report	Refund requests (1,118 in the dataset) broken down by ticket type.	Goal 2: delay and refund

2 DFD Level 0 & Level 1

SOURCES: *Team brief (Level 0 and 5 Level-1 processes); DEPI Sec. 4 "DFD - context-level and detailed levels".*

Level 0 - Context diagram

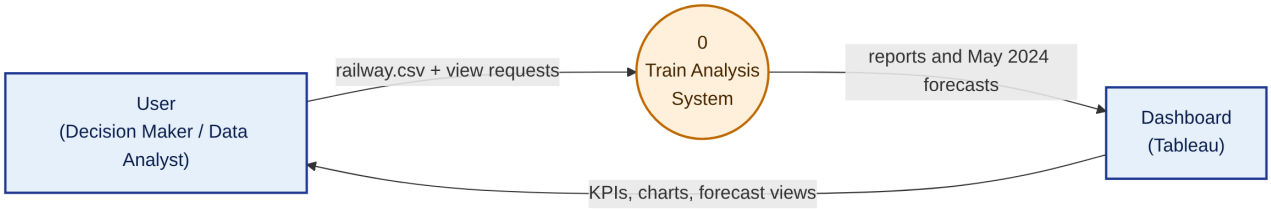


Figure 2 *- DFD Level 0: User, Train Analysis System (process 0) and Dashboard*

Mermaid code - Level 0

```
flowchart LR
  U["User<br/>(Decision Maker / Data Analyst)"]
  P0(("0<br/>Train Analysis<br/>System"))
  D["Dashboard<br/>(Tableau)"]
  U -- "railway.csv + view requests" --> P0
  P0 -- "reports and May 2024 forecasts" --> D
  D -- "KPIs, charts, forecast views" --> U
  classDef ext fill:#E8F0FE,stroke:#1F3A93,stroke-width:2px,color:#0B1F4D
  classDef proc fill:#FFF4E0,stroke:#C26A00,stroke-width:2px,color:#4A2800
  class U,D ext
  class P0 proc
```

Level 1 - Detailed processes

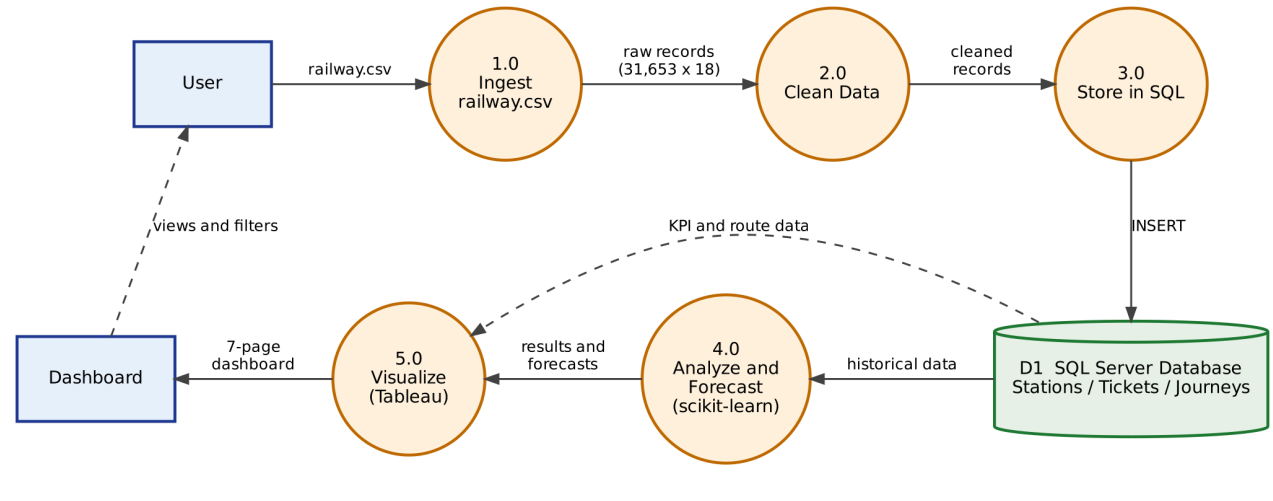


Figure 3 *- DFD Level 1 (compact layout; the Mermaid source below defines the same processes and flows)*

Mermaid code - Level 1

```
flowchart LR
  U["User"]
  P1(("1.0<br/>Ingest<br/>railway.csv"))
  P2(("2.0<br/>Clean Data"))
  P3(("3.0<br/>Store in SQL"))
  DB[("D1  SQL Server Database<br/>Stations / Tickets / Journeys")]
  P4(("4.0<br/>Analyze and Forecast<br/>scikit-learn"))
  P5(("5.0<br/>Visualize<br/>Tableau"))
  D["Dashboard"]
  U -- "railway.csv" --> P1
  P1 -- "raw records (31,653 x 18)" --> P2
  P2 -- "cleaned records" --> P3
  P3 -- "INSERT" --> DB
  DB -- "historical data" --> P4
  P4 -- "analysis results and forecasts" --> P5
  DB -- "KPI and route data" --> P5
  P5 -- "7-page dashboard" --> D
  D -- "views and filters" --> U
  classDef ext fill:#E8F0FE,stroke:#1F3A93,stroke-width:2px,color:#0B1F4D
  classDef proc fill:#FFF4E0,stroke:#C26A00,stroke-width:2px,color:#4A2800
  classDef store fill:#E6F4EA,stroke:#1E7B34,stroke-width:2px,color:#0B3D1A
  class U,D ext
  class P1,P2,P3,P4,P5 proc
  class DB store
```

Process summary

ID	Process	Input	Output	Tool
1.0	Ingest railway.csv	railway.csv (31,653 x 18)	Raw records	Pandas
2.0	Clean Data	Raw records	Cleaned records: 8 delay-reason labels merged into 5 causes, Ticket Class validated, empty Actual Arrival Time of Cancelled journeys handled	Pandas
3.0	Store in SQL	Cleaned records	Rows in Stations / Tickets / Journeys	SQL Server
4.0	Analyze & Forecast	Historical data (D1)	Analysis results; May 2024 forecasts	scikit-learn
5.0	Visualize	Results and KPI data	7-page dashboard	Tableau

3 Sequence Diagram

SOURCES: *Team brief (scenario and participants); DEPI Sec. 4 "Sequence Diagrams - key interactions between components".*

Scenario: the Decision Maker asks the dashboard for the May 2024 forecast. The dashboard queries the daily history stored in SQL Server, the data is passed to the ForecastModel, which trains, predicts and evaluates (MAPE), and the forecast is returned and displayed as a chart.

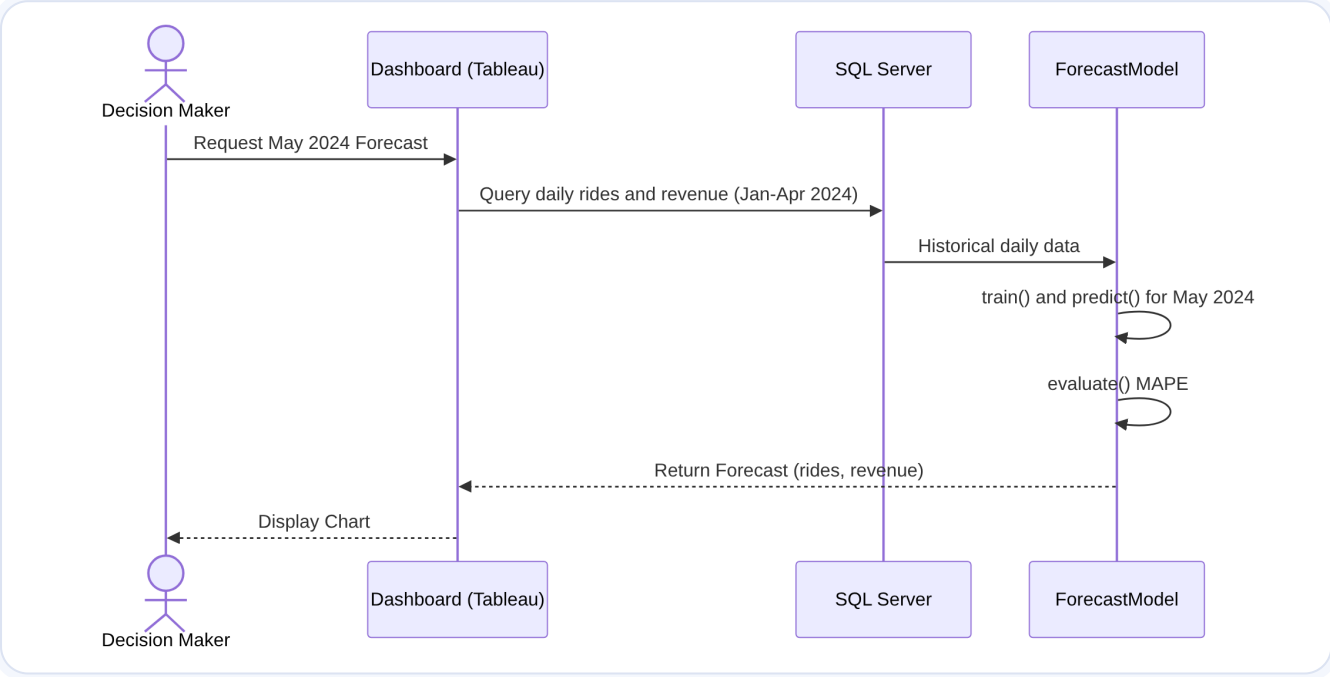


Figure 4 - *Sequence diagram: Request May 2024 Forecast*

Mermaid code

```
sequenceDiagram
  actor DM as Decision Maker
  participant DB as Dashboard (Tableau)
  participant SQL as SQL Server
  participant FM as ForecastModel
  DM->>DB: Request May 2024 Forecast
  DB->>SQL: Query daily rides and revenue (Jan-Apr 2024)
  SQL->>FM: Historical daily data
  FM->>FM: train() and predict() for May 2024
  FM->>FM: evaluate() MAPE
  FM-->>DB: Return Forecast (rides, revenue)
  DB-->>DM: Display Chart
```

Participant	Responsibility
Decision Maker	Actor who requests and reads the forecast.
Dashboard (Tableau)	Receives the request and displays the forecast chart (page 7, Forecast, of the dashboard).
SQL Server Database	Holds the cleaned Stations / Tickets / Journeys data; supplies daily rides and revenue for January - April 2024.
Forecast Model	Implements train(), predict() and evaluate(); returns the May 2024 forecast of rides and revenue.

4 Activity Diagram

SOURCES: *Team brief (activity sequence); railway.csv (data problems, verified); DEPI Sec. 4 "Activity Diagram - workflow of processes". The MAPE < 15% decision comes from the Accuracy requirement in the brief.*

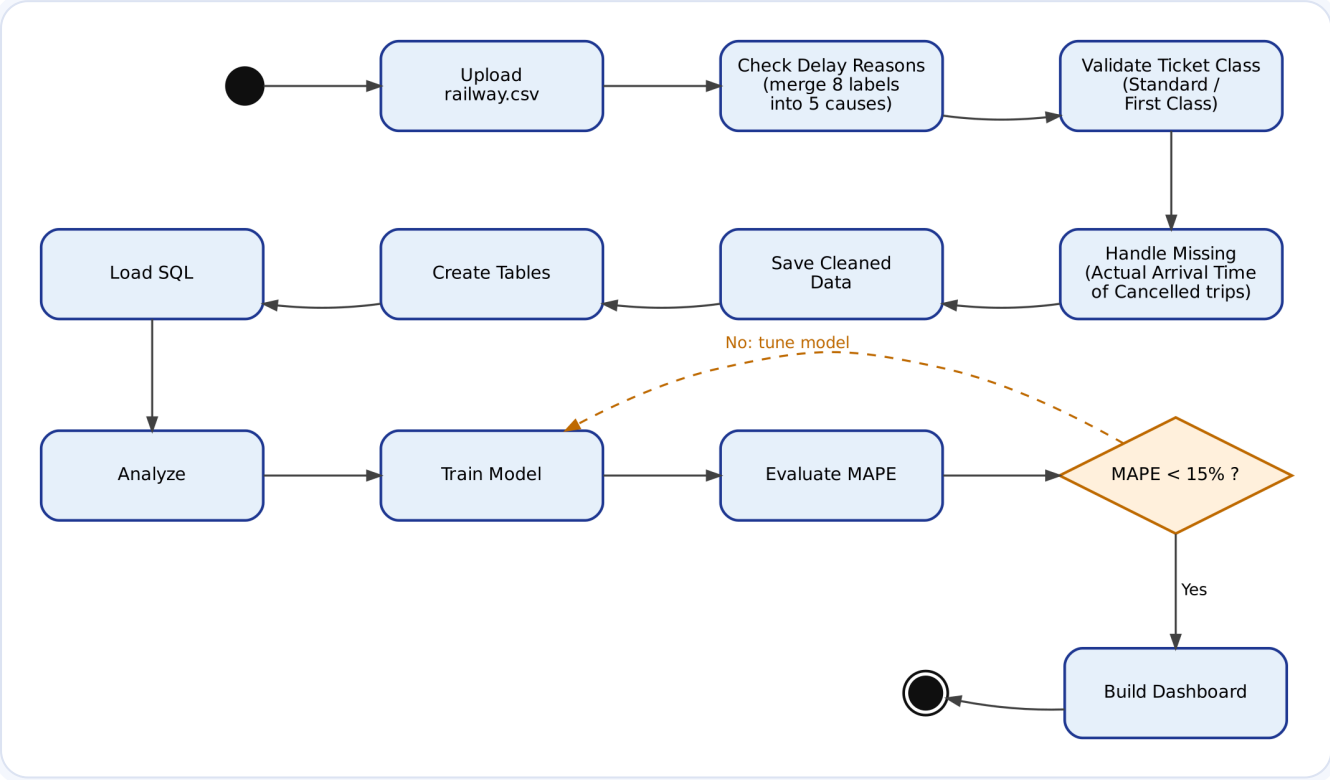


Figure 5 - *Activity diagram (compact snake layout; the Mermaid source below defines the same activities)*

Mermaid code

```
flowchart TD
  S(("Start")) --> A1["Upload railway.csv"]
  A1 --> A2["Check Delay Reasons<br/>(merge 8 labels into 5 causes)"]
  A2 --> A3["Validate Ticket Class<br/>(Standard / First Class)"]
  A3 --> A4["Handle Missing Values<br/>(Actual Arrival Time of Cancelled trips)"]
  A4 --> A5["Save Cleaned Data"]
  A5 --> A6["Create Tables"]
  A6 --> A7["Load SQL"]
  A7 --> A8["Analyze"]
  A8 --> A9["Train Model"]
  A9 --> A10["Evaluate MAPE"]
  A10 --> Q{"MAPE < 15%?"}
  Q -- "No: tune model" --> A9
  Q -- "Yes" --> A11["Build Dashboard"]
  A11 --> E((("End")))
  classDef act fill:#E8F0FE,stroke:#1F3A93,stroke-width:1.5px,color:#0B1F4D
  classDef dec fill:#FFF4E0,stroke:#C26A00,stroke-width:2px,color:#4A2800
  class A1,A2,A3,A4,A5,A6,A7,A8,A9,A10,A11 act
  class Q dec
```

- **Cleaning steps** follow the data issues found in railway.csv: Reason for Delay has 8 labels for 5 causes, and Actual Arrival Time is empty for all 1,880 Cancelled journeys. Ticket Class has only the two values Standard and First Class, so it is validated rather than corrected.

- **Decision node:** if the evaluated MAPE is not below 15%, the model is tuned and trained again; otherwise the dashboard is built.

5 State Diagram

SOURCES: *Team brief (Journey Status states); railway.csv (status and refund counts); DEPI Sec. 4 "State Diagram - states of an object and transitions".*

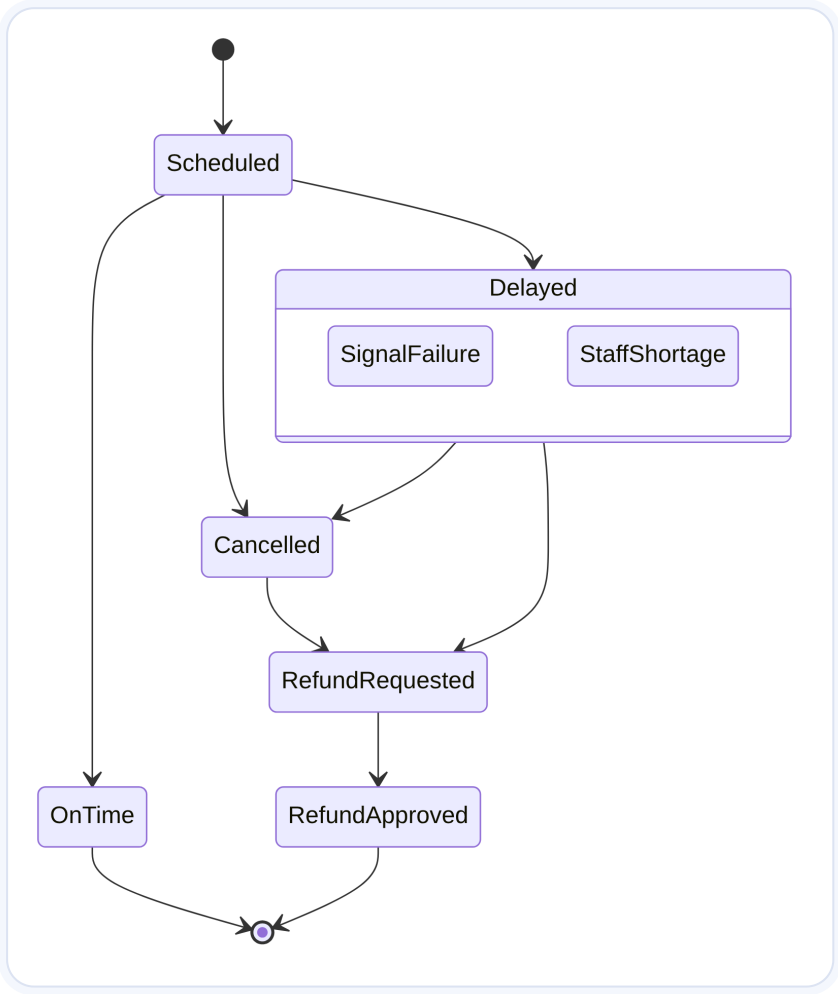


Figure 6 - *State diagram of a journey (Journey Status and refund life cycle)*

Mermaid code

```
stateDiagram-v2
  [*] --> Scheduled
  Scheduled --> OnTime
  Scheduled --> Delayed
  Scheduled --> Cancelled
  state Delayed {
    SignalFailure
    StaffShortage
  }
  Delayed --> Cancelled
  Delayed --> RefundRequested
  Cancelled --> RefundRequested
  RefundRequested --> RefundApproved
  OnTime --> [*]
  RefundApproved --> [*]
```

State	Meaning
Scheduled	Initial state of every journey.
OnTime	Journey Status = On Time (27,481 journeys); final state. No refund is requested for an On Time journey.
Delayed	Journey Status = Delayed (2,292 journeys); sub-states show delay reasons such as Signal Failure and Staff Shortage (5 causes in total).
Cancelled	Journey Status = Cancelled (1,880 journeys); Actual Arrival Time is empty.
RefundRequested	Refund Request = Yes: 1,118 requests (546 on Delayed and 572 on Cancelled journeys).
RefundApproved	Refund granted; final state. Modelled state: railway.csv records only whether a refund was requested, not its outcome.

Note: in railway.csv refund requests occur on both Delayed (546) and Cancelled (572) journeys and never on On Time journeys, which matches the transitions to Refund Requested shown in the diagram.

6 Class Diagram

SOURCES: *Team brief (4 classes and members); DEPI Sec. 4 "Class Diagram - classes, attributes, methods, relationships".*

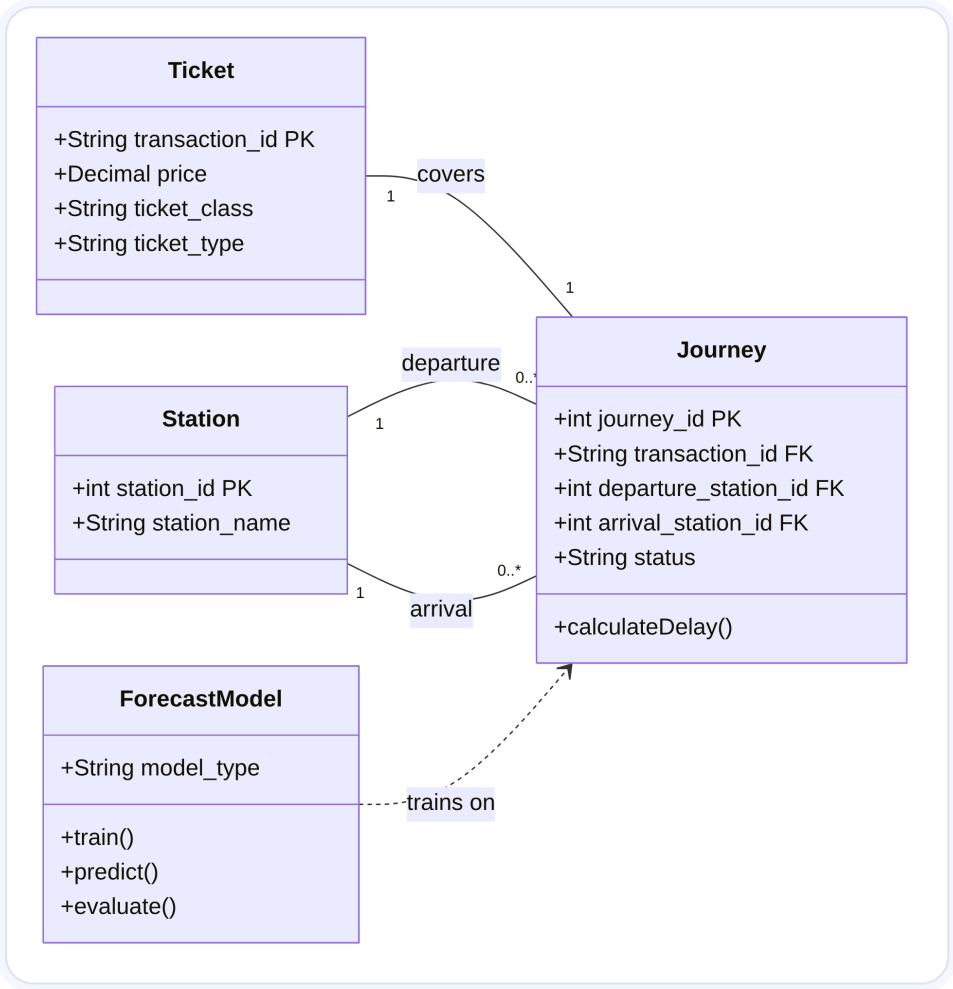


Figure 7 *- Class diagram*

Mermaid code

```
classDiagram
  direction LR
  class Ticket {
    +String transaction_id PK
    +Decimal price
    +String ticket_class
    +String ticket_type
  }
  class Journey {
    +int journey_id PK
    +String transaction_id FK
    +int departure_station_id FK
    +int arrival_station_id FK
    +String status
    +calculateDelay()
  }
  class Station {
    +int station_id PK
    +String station_name
  }
  class ForecastModel {
    +String model_type
    +train()
    +predict()
    +evaluate()
  }
  Ticket "1" -- "1" Journey : covers
  Station "1" -- "0..*" Journey : departure
  Station "1" -- "0..*" Journey : arrival
  ForecastModel ..> Journey : trains on
```

- **Ticket (1) - (1) Journey:** one ticket (Transaction ID) covers exactly one journey.
- **Station (1) - (0..*) Journey:** a station can be the departure of many journeys and the arrival of many journeys, so Journey holds departure_station_id and arrival_station_id (the brief lists only departure_station_id; arrival_station_id is added to match the logical schema).
- **calculateDelay():** Actual Arrival Time minus scheduled Arrival Time, counted across midnight for the 961 journeys that arrive on the next day; undefined (NULL) for Cancelled journeys.
- **ForecastModel:** model_type is Random Forest or Prophet; train(), predict() and evaluate() (MAPE).

7 ER Diagram, Logical Schema and Physical Schema

SOURCES: *Team brief (ER cardinalities, logical tables, constraint types); DEPI Sec. 4 "ER Diagram" and "Logical & Physical Schema"; railway.csv and railway_data_dictionary.csv (columns, values and constraints checked against all 31,653 rows).*

7.1 ER Diagram

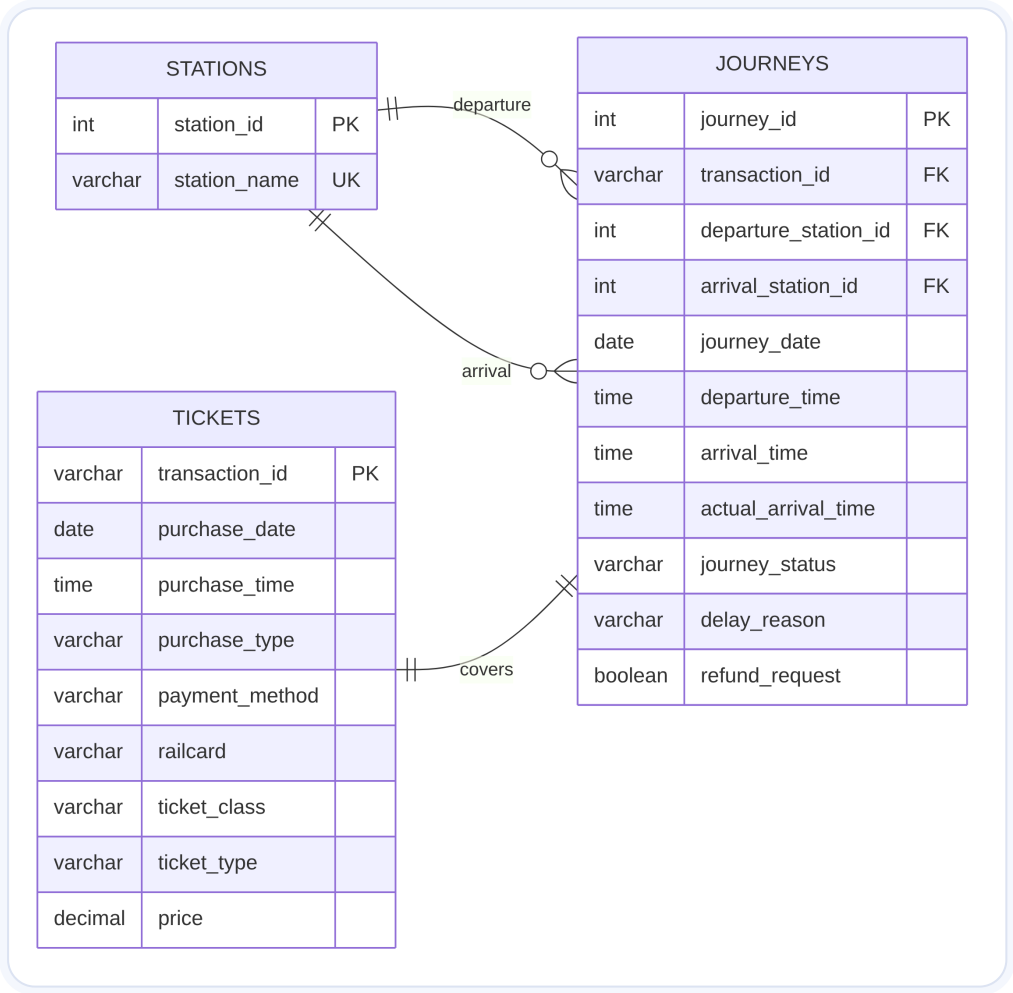


Figure 8 *- ER diagram: Station (1) - (N) Journey (departure and arrival); Ticket (1) - (1) Journey*

The ER diagram source is delivered as er.mmd in the source bundle for GitHub.

7.2 Logical Schema

Normalised to 3NF: station names are stored once in Stations; ticket attributes depend only on the ticket; journey attributes depend only on the journey. The columns marked (+) are added to the brief's logical schema so that all 18 columns of railway.csv are stored and rides per day can be computed.

Stations	Type	Key	Source column in railway.csv
station_id	INT	PK	Generated (surrogate)
station_name	VARCHAR(100)	UNIQUE	Departure Station and Arrival Destination (32 distinct values)

Tickets	Type	Key	Source column in railway.csv
transaction_id	VARCHAR(50)	PK	Transaction ID
purchase_date	DATE		Date of Purchase
purchase_time (+)	TIME		Time of Purchase
purchase_type (+)	VARCHAR(20)		Purchase Type
payment_method	VARCHAR(30)		Payment Method
railcard	VARCHAR(30)		Railcard
ticket_class	VARCHAR(20)		Ticket Class (Standard / First Class)
ticket_type	VARCHAR(30)		Ticket Type
price	DECIMAL(8,2)		Price

Journeys	Type	Key	Source column in railway.csv
journey_id	INT	PK	Generated (surrogate)
transaction_id	VARCHAR(50)	FK, UNIQUE	Transaction ID -> Tickets
departure_station_id	INT	FK	Departure Station -> Stations
arrival_station_id	INT	FK	Arrival Destination -> Stations
journey_date (+)	DATE		Date of Journey
departure_time (+)	TIME		Departure Time
arrival_time (+)	TIME		Arrival Time
actual_arrival_time (+)	TIME NULL		Actual Arrival Time (empty when Cancelled)
journey_status	VARCHAR(20)		Journey Status
delay_reason	VARCHAR(50) NULL		Reason for Delay (8 labels merged into 5 causes)
refund_request	BIT		Refund Request (Yes/No converted to 1/0 in step 2.0)

7.3 Physical Schema (SQL Server, T-SQL)

Ready-to-run T-SQL script for SQL Server (SSMS); it uses DROP … IF EXISTS and CREATE OR ALTER, so SQL Server 2016 SP1 or later is needed. Run the batches (separated by GO) in order. The CHECK lists use the values found in railway.csv after cleaning (delay reasons merged into 5 causes; Refund Request converted from Yes/No to 1/0). The constraint logic was checked by loading all 31,653 cleaned rows (32 stations) into an equivalent schema. The loading script (staging table, cleaning rules and a verification query) is 02_load_data.sql in the source bundle.

```sql
-- 1. Database
IF DB_ID(N'uk_train_rides') IS NULL
    CREATE DATABASE uk_train_rides;
GO

USE uk_train_rides;
GO

-- 2. Re-run safety: drop in dependency order
DROP VIEW IF EXISTS dbo.v_journey_details;
DROP VIEW IF EXISTS dbo.v_daily_summary;
DROP VIEW IF EXISTS dbo.v_route_revenue;
DROP TABLE IF EXISTS dbo.journeys;
DROP TABLE IF EXISTS dbo.tickets;
DROP TABLE IF EXISTS dbo.stations;
GO

-- 3. Stations (32 distinct names in the data)
CREATE TABLE dbo.stations
(
    station_id INT IDENTITY(1,1) NOT NULL, -- surrogate key
    station_name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_stations PRIMARY KEY (station_id),
    CONSTRAINT uq_stations_name UNIQUE (station_name)
);
GO

-- 4. Tickets (one row per Transaction ID)
CREATE TABLE dbo.tickets
(
    transaction_id VARCHAR(50) NOT NULL, -- 23 characters in the data
    purchase_date DATE NOT NULL,
    purchase_time TIME(0) NOT NULL,
    purchase_type VARCHAR(20) NOT NULL,
    payment_method VARCHAR(30) NOT NULL,
    railcard VARCHAR(30) NOT NULL
        CONSTRAINT df_tickets_railcard DEFAULT 'None', -- 'None' is a real value
    ticket_class VARCHAR(20) NOT NULL,
    ticket_type VARCHAR(30) NOT NULL,
    price DECIMAL(8,2) NOT NULL,
    CONSTRAINT pk_tickets PRIMARY KEY (transaction_id),
    CONSTRAINT chk_tickets_price CHECK (price >= 0),
    CONSTRAINT chk_tickets_class
        CHECK (ticket_class IN ('Standard', 'First Class')),
    CONSTRAINT chk_tickets_type
        CHECK (ticket_type IN ('Advance', 'Off-Peak', 'Anytime')),
    CONSTRAINT chk_tickets_purchase
        CHECK (purchase_type IN ('Online', 'Station')),
    CONSTRAINT chk_tickets_payment
        CHECK (payment_method IN ('Contactless', 'Credit Card', 'Debit Card')),
    CONSTRAINT chk_tickets_railcard
        CHECK (railcard IN ('None', 'Adult', 'Senior', 'Disabled'))
);
GO

-- 5. Journeys (one row per ticket)
CREATE TABLE dbo.journeys
(
```

```
    journey_id INT IDENTITY(1,1) NOT NULL,
    transaction_id VARCHAR(50) NOT NULL,
    departure_station_id INT NOT NULL,
    arrival_station_id INT NOT NULL,
    journey_date DATE NOT NULL,
    departure_time TIME(0) NOT NULL,
    arrival_time TIME(0) NOT NULL, -- can fall on the next day
    actual_arrival_time TIME(0) NULL, -- empty when Cancelled
    journey_status VARCHAR(20) NOT NULL,
    delay_reason VARCHAR(50) NULL, -- merged into 5 causes
    refund_request BIT NOT NULL
        CONSTRAINT df_journeys_refund DEFAULT 0, -- Yes = 1, No = 0
    CONSTRAINT pk_journeys PRIMARY KEY (journey_id),
    CONSTRAINT uq_journeys_transaction UNIQUE (transaction_id), -- 1 ticket : 1 journey
    CONSTRAINT fk_journeys_ticket FOREIGN KEY (transaction_id)
        REFERENCES dbo.tickets (transaction_id)
        ON DELETE NO ACTION ON UPDATE NO ACTION,
    CONSTRAINT fk_journeys_departure FOREIGN KEY (departure_station_id)
        REFERENCES dbo.stations (station_id)
        ON DELETE NO ACTION ON UPDATE NO ACTION,
    CONSTRAINT fk_journeys_arrival FOREIGN KEY (arrival_station_id)
        REFERENCES dbo.stations (station_id)
        ON DELETE NO ACTION ON UPDATE NO ACTION,
    CONSTRAINT chk_journeys_status
        CHECK (journey_status IN ('On Time', 'Delayed', 'Cancelled')),
    CONSTRAINT chk_journeys_stations
        CHECK (departure_station_id <> arrival_station_id),
    CONSTRAINT chk_journeys_cancelled
        CHECK (
            (journey_status = 'Cancelled' AND actual_arrival_time IS NULL)
            OR (journey_status <> 'Cancelled' AND actual_arrival_time IS NOT NULL)
        ),
    CONSTRAINT chk_journeys_reason
        CHECK (
            (journey_status = 'On Time' AND delay_reason IS NULL)
            OR (
                journey_status <> 'On Time'
                AND delay_reason IN (
                    'Weather', 'Signal Failure', 'Staff Shortage',
                    'Technical Issue', 'Traffic'
                )
            )
        ),
    CONSTRAINT chk_journeys_refund
        CHECK (refund_request = 0 OR journey_status <> 'On Time')
);
GO

-- 6. Indexes for the dashboard queries
CREATE INDEX ix_journeys_date
    ON dbo.journeys (journey_date);
CREATE INDEX ix_journeys_route
    ON dbo.journeys (departure_station_id, arrival_station_id);
GO

-- 7. Views used by the dashboard (flat view for Tableau) and the forecast
CREATE OR ALTER VIEW dbo.v_route_revenue
AS
SELECT
    dep.station_name AS departure_station,
    arr.station_name AS arrival_station,
    COUNT(*) AS rides,
    SUM(t.price) AS revenue
FROM dbo.journeys AS j
JOIN dbo.tickets AS t
    ON t.transaction_id = j.transaction_id
JOIN dbo.stations AS dep
    ON dep.station_id = j.departure_station_id
JOIN dbo.stations AS arr
    ON arr.station_id = j.arrival_station_id
GROUP BY
    dep.station_name,
```

```
    arr.station_name;
GO

CREATE OR ALTER VIEW dbo.v_daily_summary
AS
SELECT
    j.journey_date,
    COUNT(*) AS rides,
    SUM(t.price) AS revenue
FROM dbo.journeys AS j
JOIN dbo.tickets AS t
    ON t.transaction_id = j.transaction_id
GROUP BY
    j.journey_date;
GO

CREATE OR ALTER VIEW dbo.v_journey_details
AS
SELECT
    j.journey_id,
    j.journey_date,
    dep.station_name AS departure_station,
    arr.station_name AS arrival_station,
    j.departure_time,
    j.arrival_time,
    j.actual_arrival_time,
    CASE -- signed minutes, correct across midnight
        WHEN j.actual_arrival_time IS NULL THEN NULL
        ELSE (((DATEDIFF(MINUTE, j.arrival_time, j.actual_arrival_time) + 720) % 1440) + 1440) % 1440 - 720
    END AS delay_minutes,
    j.journey_status,
    j.delay_reason,
    j.refund_request,
    t.transaction_id,
    t.purchase_date,
    t.purchase_time,
    t.purchase_type,
    t.payment_method,
    t.railcard,
    t.ticket_class,
    t.ticket_type,
    t.price
FROM dbo.journeys AS j
JOIN dbo.tickets AS t
    ON t.transaction_id = j.transaction_id
JOIN dbo.stations AS dep
    ON dep.station_id = j.departure_station_id
JOIN dbo.stations AS arr
    ON arr.station_id = j.arrival_station_id;
GO
```

8 Wireframes and Technology Stack

SOURCES: *Team brief (initial outline of 4 Tableau pages and the tools, extended to 7 pages in this version); railway.csv (every KPI value and chart value shown in the wireframes, computed over all 31,653 rows); DEPI Sec. 4 "Wireframes & Mockups" and "Technology Stack"; DEPI note "All documents should be uploaded to GitHub".*

8.1 Wireframes - 7 Tableau pages

Every page has the same structure: a navigation bar with the 7 page tabs and one shared date filter (Date of Journey, 1 Jan - 30 Apr 2024), then a row of four large KPI cards, then 4-5 visuals. Each visual is titled with its chart type and carries the business question it answers. The four KPI cards are the same on every page: Total Revenue (sum of Price), Total Rides (count of Transaction ID), On-Time % (share of journeys with Journey Status = On Time) and Refunds (count of Refund Request = Yes).

The project brief outlined 4 Tableau pages; the dashboard was extended to 7 so that every question about routes, revenue, punctuality, delays, refunds, customers and the May 2024 forecast has a place. All numbers in the wireframes are computed from railway.csv; the May 2024 forecast areas and the forecast columns are placeholders for the model output. Station names are shortened in the charts (for example "Euston" for London Euston, "Manchester" for Manchester Piccadilly).

Page	Name	Main question	Use cases / requirements
1	Overview	What happened in the period overall?	All use cases (summary); FR8
2	Routes	Where do passengers travel and which routes earn most?	UC1; FR4
3	Revenue	Which tickets, classes and railcards bring the money?	UC1, UC5 (actual class revenue); FR4
4	Punctuality	How punctual is the service and where does it fail?	Context for UC2
5	Delays & Refunds	Why do journeys fail and who asks for refunds?	UC2, UC6; FR5
6	Customers	How and when do passengers buy and travel?	Supporting analysis (no new use case)
7	Forecast	What to expect in May 2024?	UC3, UC4, UC5; FR6, FR7

Page 1 - Overview

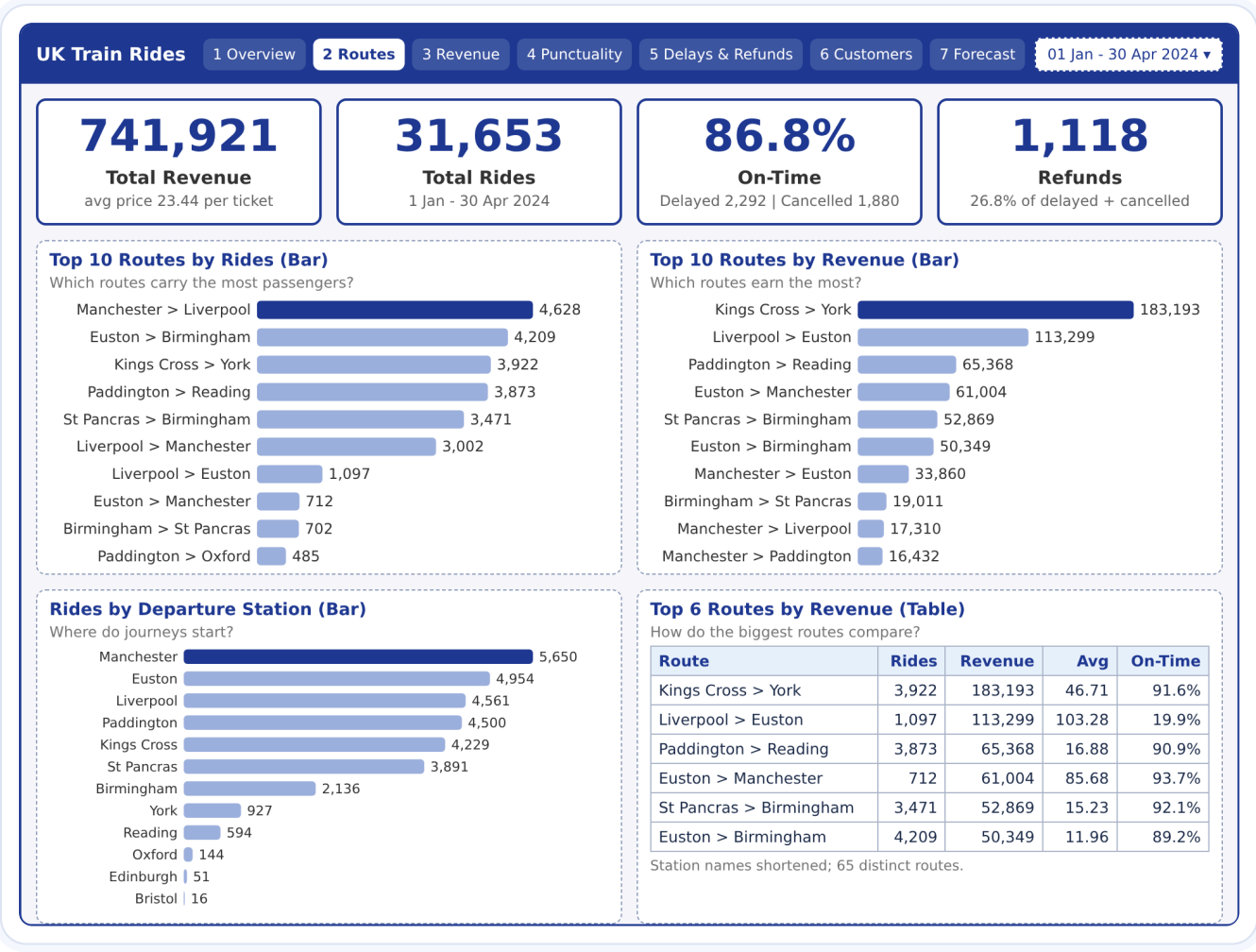


Month	Rides	Revenue	On-Time	Refunds
Jan	8,111	199,618	87.3%	289
Feb	7,644	159,374	87.0%	253
Mar	8,117	195,147	86.0%	306
Apr	7,781	187,782	87.0%	270
Total	**31,653**	**741,921**	**86.8%**	**1,118**

Wireframe - page 1 of 7 : *Overview (values computed from railway.csv).*

Purpose	Executive summary for the Decision Maker: the four KPIs, the daily trend and the monthly comparison on one page.
Questions answered	How much money and how many rides? How punctual is the network? How many refunds? Is there a trend or a bad day? Which month was best?
Visuals and fields	4 KPI cards; Daily Revenue (line); Daily Rides (line); Monthly Summary (table); Journey Status (donut). Fields: Price, Transaction ID, Date of Journey, Journey Status, Refund Request.
Key findings (railway.csv)	Revenue 741,921 from 31,653 rides, an average of 23.44 per ticket. Daily rides average 261.6 (minimum 63, maximum 313 on 9 Mar 2024); daily revenue averages 6,131.6 (range 1,562 - 9,196). The three lowest days by rides are 1 Apr 2024 (63), 1 Jan 2024 (66) and 1 Mar 2024 (73); all three are the first day of a month. January earns the most (199,618); February the least (159,374 from 7,644 rides). 86.8% of journeys are on time; 2,292 are Delayed (7.2%) and 1,880 Cancelled (5.9%).

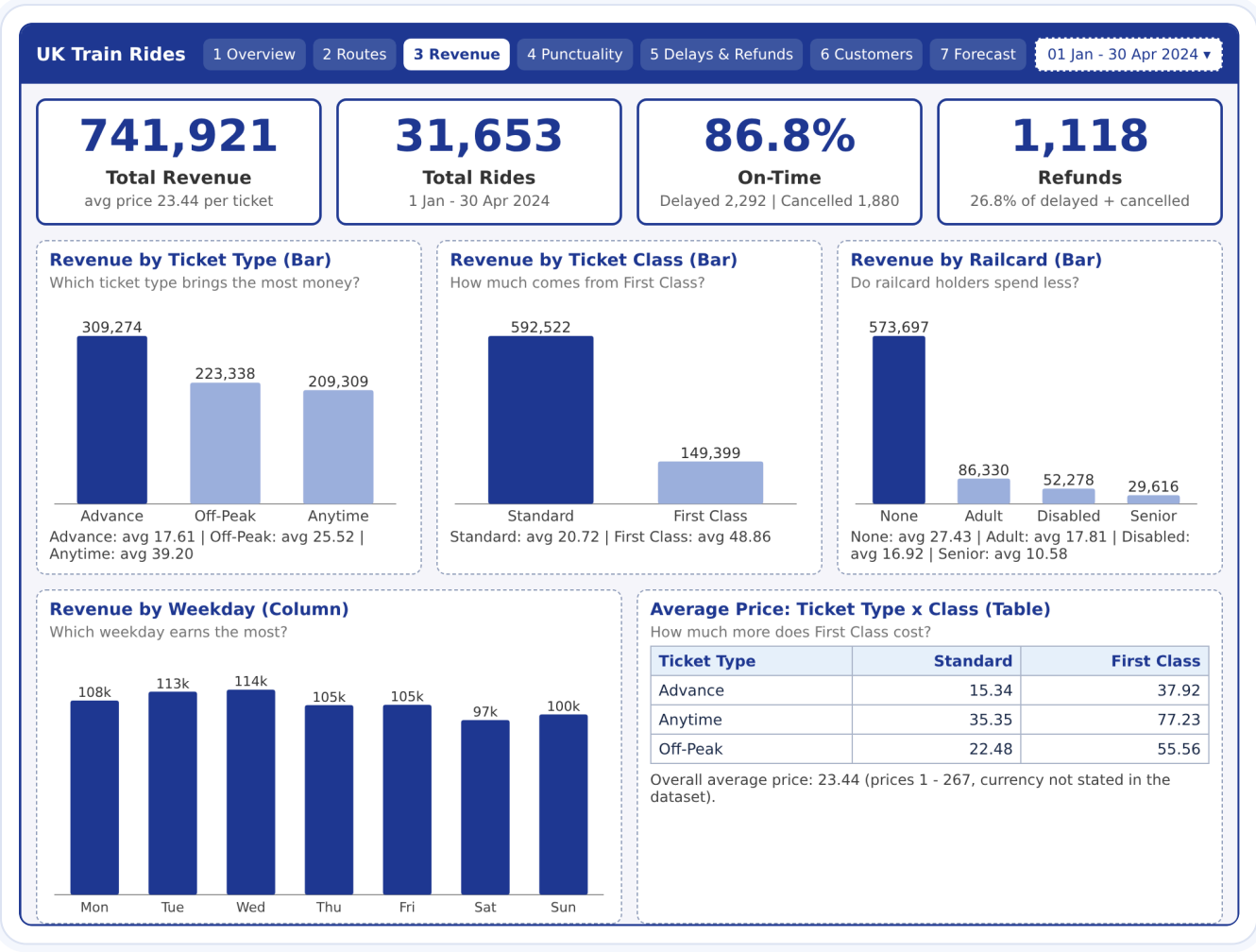
Page 2 - Routes



Wireframe - page 2 of 7 : *Routes (values computed from railway.csv).*

Purpose	Show where demand and money are concentrated across the 65 distinct routes and 12 departure stations.
Questions answered	Which routes carry the most passengers? Which earn the most? Where do journeys start? How do the biggest routes compare on price and punctuality?
Visuals and fields	Top 10 Routes by Rides (bar); Top 10 Routes by Revenue (bar); Rides by Departure Station (bar); Top 6 Routes by Revenue (table). Fields: Departure Station, Arrival Destination, Price, Journey Status.
Key findings (railway.csv)	Manchester Piccadilly - Liverpool Lime Street (4,628 rides) is the busiest route but earns only 17,310 (average price 3.74). London Euston - Birmingham New Street is 2nd with 4,209 rides. London Kings Cross - York earns the most: 183,193 from 3,922 rides (average price 46.71). Liverpool Lime Street - London Euston is 2nd by revenue (113,299) from only 1,097 rides (average price 103.28). Manchester Piccadilly is the busiest departure station (5,650 rides), followed by London Euston (4,954); Bristol Temple Meads has only 16.

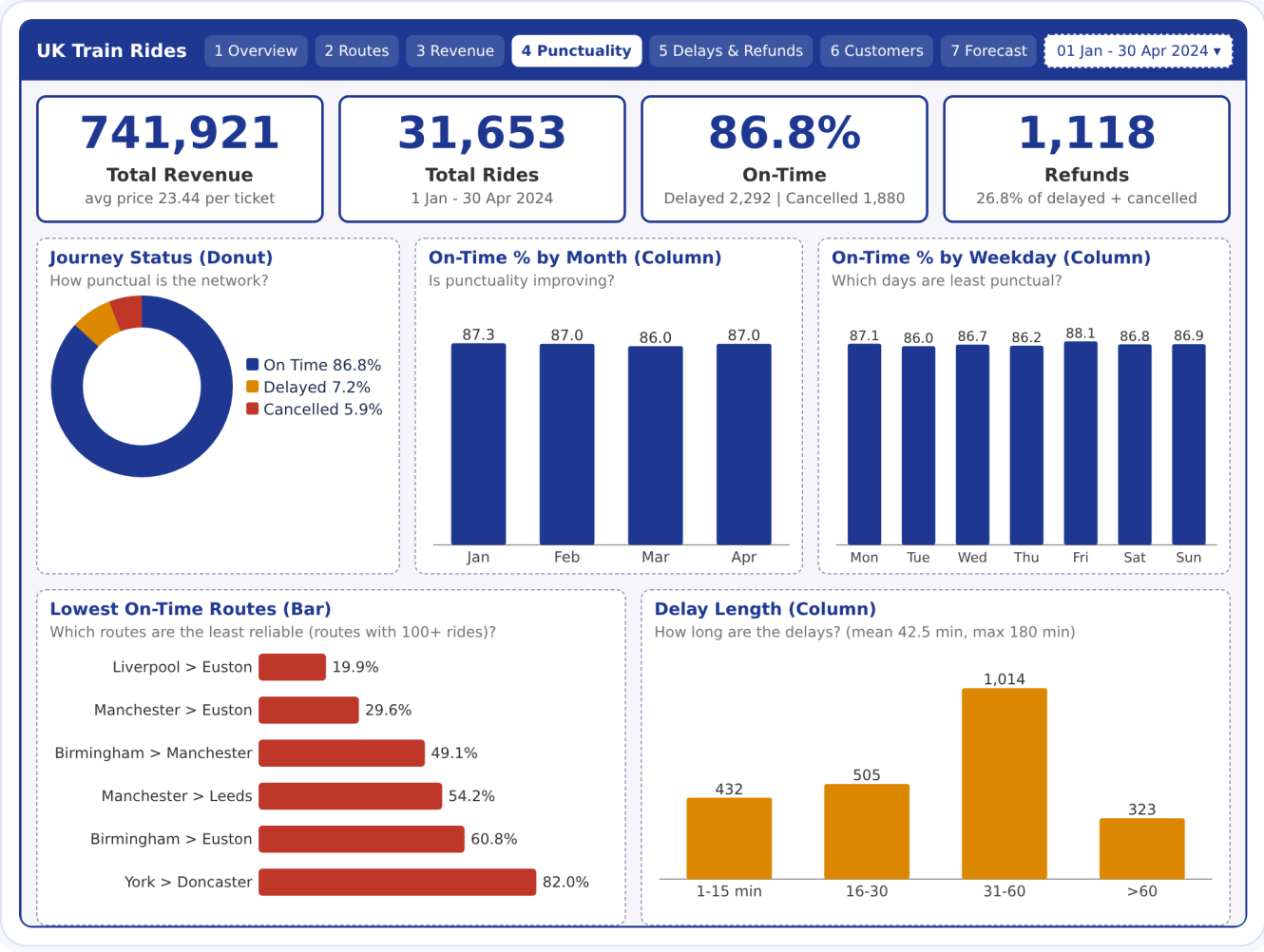
Page 3 - Revenue



Wireframe - page 3 of 7 : *Revenue (values computed from railway.csv).*

Purpose	Explain what drives revenue: ticket type, ticket class, railcard and weekday, and how much each group pays on average.
Questions answered	Which ticket type brings the most money? How much comes from First Class? Do railcard holders pay less? Which weekday earns most? How much more does First Class cost?
Visuals and fields	Revenue by Ticket Type, Ticket Class and Railcard (bars with average price); Revenue by Weekday (column); Average Price by Ticket Type x Class (table). Fields: Price, Ticket Type, Ticket Class, Railcard, Date of Journey.
Key findings (railway.csv)	Advance tickets bring 309,274 (41.7% of revenue) at the lowest average price (17.61); Anytime has the highest average (39.20). First Class is 9.7% of rides but 20.1% of revenue (149,399); average price 48.86 against 20.72 for Standard. Passengers without a railcard pay 27.43 on average; Senior railcard holders 10.58, Disabled 16.92 and Adult 17.81. Wednesday earns the most (113,802) and Saturday the least (96,936). The currency of Price is not stated in the dataset.

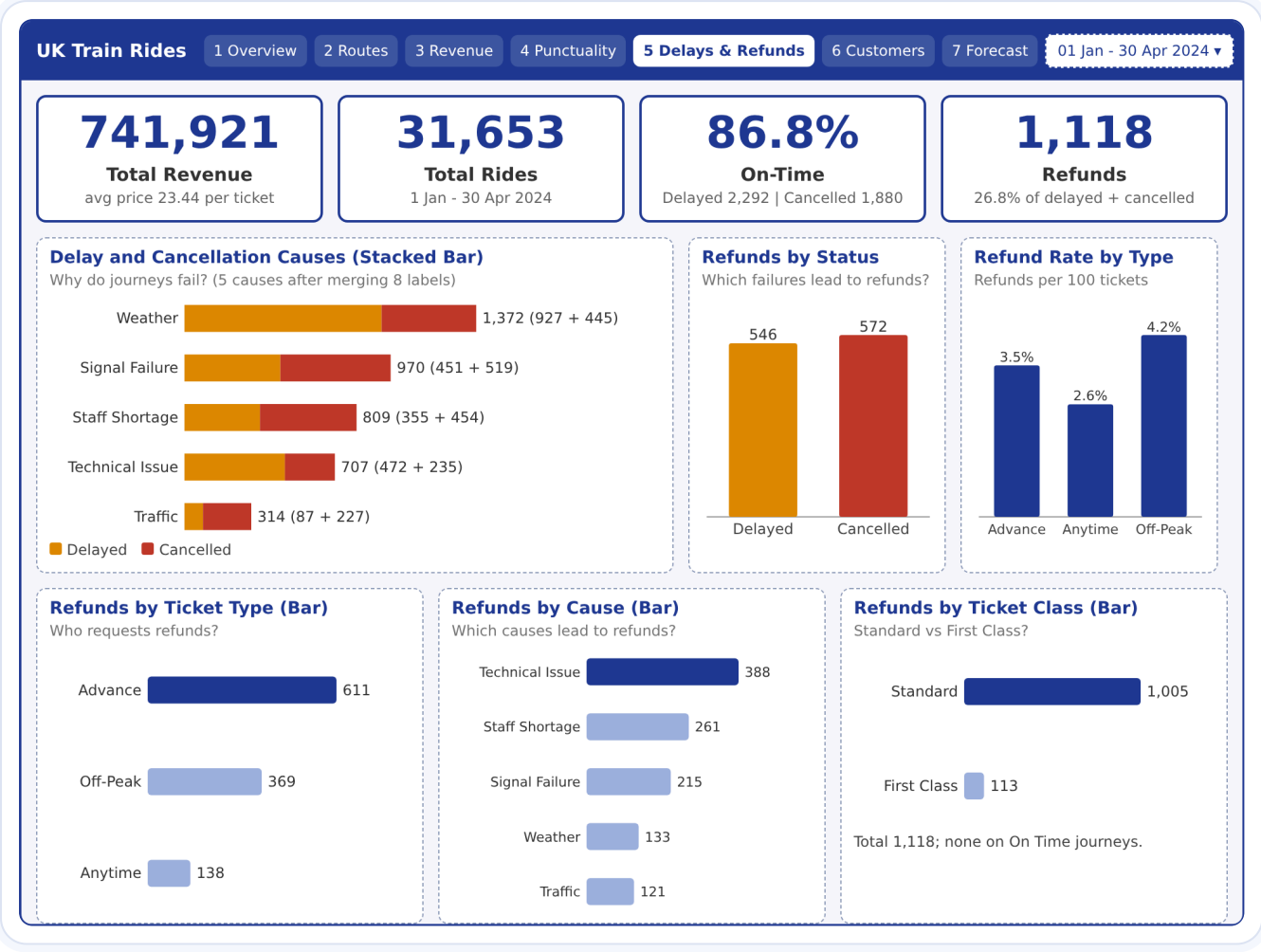
Page 4 - Punctuality



Wireframe - page 4 of 7 : *Punctuality (values computed from railway.csv).*

Purpose	Measure punctuality over time and by route, and show how long delays last.
Questions answered	How punctual is the network? Is it improving? Which days and routes are the least reliable? How long are the delays?
Visuals and fields	Journey Status (donut); On-Time % by Month (column); On-Time % by Weekday (column); Lowest On-Time Routes (bar, routes with 100+ rides); Delay Length (column). Fields: Journey Status, Date of Journey, Arrival Time, Actual Arrival Time.
Key findings (railway.csv)	On-time share is stable: 86.0% (Mar) to 87.3% (Jan) by month and 86.0% (Tuesday) to 88.1% (Friday) by weekday. Two routes stand out: Liverpool Lime Street - London Euston is on time only 19.9% of the time (1,097 rides) and Manchester Piccadilly - London Euston 29.6% (345 rides). 34 routes have 100 or more rides. 2,274 Delayed journeys have a positive delay (the other 18 Delayed rows are not later than scheduled): mean 42.5 min, median 37 min, maximum 180 min. The 31-60 minute band is the largest (1,014 journeys) and 323 delays exceed 60 minutes.

Page 5 - Delays & Refunds



Wireframe - page 5 of 7 : *Delays & Refunds (values computed from railway.csv).*

Purpose	Show the causes of Delayed and Cancelled journeys and where refund requests come from.
Questions answered	Why do journeys fail? Which failures lead to refunds? Which ticket type, class and cause generate the most refund requests?
Visuals and fields	Delay and Cancellation Causes (stacked bar); Refunds by Status (column); Refund Rate by Type (column); Refunds by Ticket Type, Cause and Class (bars). Fields: Reason for Delay (5 causes), Journey Status, Refund Request, Ticket Type, Ticket Class.
Key findings (railway.csv)	Weather is the largest cause: 1,372 of the 4,172 failed journeys (927 delayed + 445 cancelled), then Signal Failure 970, Staff Shortage 809, Technical Issue 707 and Traffic 314. 1,118 refunds were requested (26.8% of Delayed + Cancelled): 546 on Delayed and 572 on Cancelled journeys, none on On Time journeys. By ticket type: Advance 611, Off-Peak 369, Anytime 138; the refund rate per ticket is highest for Off-Peak (4.2%) and lowest for Anytime (2.6%). Technical Issue produces the most refund requests (388) although Weather is the largest cause; Standard passengers make 1,005 requests and First Class 113.

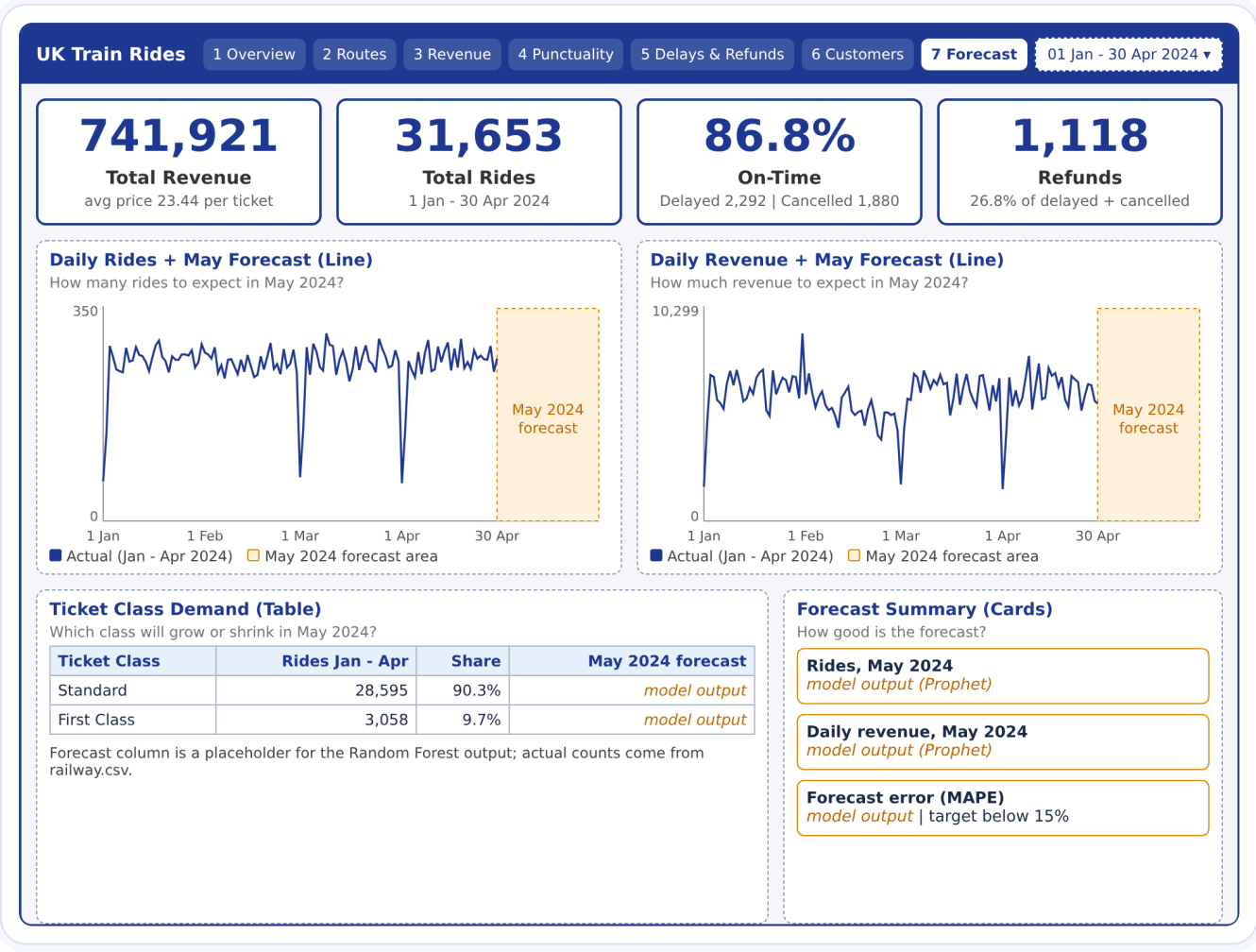
Page 6 - Customers



Wireframe - page 6 of 7 : *Customers (values computed from railway.csv).*

Purpose	Describe how passengers buy and when they travel, to support operational and commercial planning.
Questions answered	Do passengers buy online or at the station? How do they pay? Who uses a railcard? When are the travel peaks? How early do they buy?
Visuals and fields	Purchase Channel (donut); Payment Method (bar); Railcard Holders (bar); Rides by Departure Hour (column); Booking Lead Time (bar). Fields: Purchase Type, Payment Method, Railcard, Departure Time, Date of Purchase, Date of Journey.
Key findings (railway.csv)	Online 18,521 (58.5%) against Station 13,132 (41.5%). Credit Card 19,136, Contactless 10,834, Debit Card 1,683; 66.1% of tickets have no railcard (the value is the text None). Departures peak at 06:00-08:59 (hours 6 and 7 carry 3,112 and 2,795 rides) and at 16:00-18:59 (hour 18 is the busiest with 3,113). 87.9% of tickets are bought 0-1 days before the journey; tickets bought earlier are all Advance tickets, up to 28 days ahead.

Page 7 - Forecast



Wireframe - page 7 of 7 : *Forecast (values computed from railway.csv).*

Purpose	Present the May 2024 forecast next to the January - April history, with the model quality target.
Questions answered	How many rides and how much revenue to expect in May 2024? Which ticket class will grow or shrink? How accurate is the forecast?
Visuals and fields	Daily Rides + May Forecast (line); Daily Revenue + May Forecast (line); Ticket Class Demand (table); Forecast Summary (cards). Models: Prophet for the daily series, Random Forest for ticket class demand.
Key findings (railway.csv)	History: 121 daily observations (1 Jan - 30 Apr 2024); the shaded May area (31 days) is a placeholder for the model output. Ticket class baseline: Standard 28,595 rides (90.3%) and First Class 3,058 (9.7%); the May 2024 column is a placeholder for the Random Forest output. Quality target: forecast MAPE below 15% (NFR3), evaluated on a hold-out period. The four KPI cards at the top show the historical totals and do not change with the forecast.

8.2 Technology Stack

Layer	Tool	Justification
Storage	SQL Server	Relational model with PRIMARY KEY, FOREIGN KEY, UNIQUE and CHECK constraints for the Stations / Tickets / Journeys schema; three SQL views (daily summary, route revenue, flat journey details) feed the dashboard.
Cleaning	Pandas	Merges the 8 delay-reason labels into 5 causes, validates Ticket Class and station names, and handles the empty Actual Arrival Time of Cancelled journeys.
Forecasting	scikit-learn (Random Forest) + Prophet	Random Forest for tabular demand prediction such as ticket class demand; Prophet for daily time series (rides and revenue) for May 2024. The model-to-target mapping is the proposed design.
Visualization	Tableau	Builds the 7-page interactive dashboard: the same 4 KPI cards at the top of every page, one shared date filter, and bar, stacked bar, column, line, donut and table visuals. Pages 3-6 read the flat view v_journey_details (it also supplies delay_minutes); pages 1-2 can use v_daily_summary and v_route_revenue.
Version Control	GitHub	DEPI requires all documents to be uploaded to GitHub; a private repository also satisfies the security requirement.

9 Data Dictionary - railway.csv

SOURCES: *railway_data_dictionary.csv (field descriptions); railway.csv (data types, examples, counts and issues, computed over all 31,653 rows); Team brief (18 column names).*

31,653 rows x 18 columns; journeys from 1 Jan to 30 Apr 2024 (purchases from 8 Dec 2023). Examples are taken from the first two rows of railway.csv. Data types are the proposed SQL Server types.

Column Name	Data Type	Example	Description	Issue
Transaction ID	VARCHAR(50)	da8a6ba8-b3dc-4677-b176	Unique identifier for an individual train ticket purchase.	None: 31,653 unique values, all 23 characters.
Date of Purchase	DATE	2023-12-08	Date the ticket was purchased.	Range 8 Dec 2023 - 30 Apr 2024: purchases start before the journey period.
Time of Purchase	TIME	12:41:11	Time the ticket was purchased.	None
Purchase Type	VARCHAR(20)	Online	Whether the ticket was purchased online or directly at a station.	None (Online 18,521; Station 13,132).
Payment Method	VARCHAR(30)	Contactless	Payment method used: Contactless, Credit Card or Debit Card.	None (the source dictionary spells it "Contactles").
Railcard	VARCHAR(30)	Adult	National Railcard holder (Adult, Senior or Disabled) or not (None); holders get 1/3 off.	'None' is a real value (20,918 rows); pandas treats it as missing unless keep_default_na=False.
Ticket Class	VARCHAR(20)	Standard	Seat class: Standard (28,595) or First Class (3,058).	None: only 2 values; First Class is 9.7% of tickets (class imbalance).
Ticket Type	VARCHAR(30)	Advance	When the ticket was bought or can be used: Advance, Off-Peak or Anytime.	None
Price	DECIMAL(8,2)	43	Final cost of the ticket; basis of revenue (total 741,921).	None: whole numbers from 1 to 267; currency not stated.
Departure Station	VARCHAR(100)	London Paddington	Station to board the train.	12 distinct stations.
Arrival Destination	VARCHAR(100)	Liverpool Lime Street	Station to exit the train.	32 distinct values; 'Edinburgh' (154) and 'Edinburgh Waverley' (178) may be the same station.
Date of Journey	DATE	2024-01-01	Date the train departed.	None: 1 Jan - 30 Apr 2024 (121 days).
Departure Time	TIME	11:00:00	Time the train departed.	None
Arrival Time	TIME	13:30:00	Time the train was scheduled to arrive (can be on the day after departure).	961 journeys arrive after midnight (Arrival Time earlier than Departure Time).
Actual Arrival Time	TIME (NULL)	11:40:00	Time the train actually arrived (can be on the day after departure).	Empty for all 1,880 Cancelled journeys.
Journey Status	VARCHAR(20)	Delayed	Whether the train was On Time, Delayed or Cancelled.	On Time 27,481 (86.8%); Delayed 2,292; Cancelled 1,880. 18 Delayed journeys have an Actual Arrival Time not later than scheduled.
Reason for Delay	VARCHAR(50)	Signal Failure	Reason for the delay or cancellation.	8 labels for 5 causes (Signal Failure / Signal failure, Staff Shortage / Staffing, Weather / Weather Conditions, Technical Issue, Traffic); empty for On Time journeys.
Refund Request	BIT	No	Whether the passenger requested a refund after a delay or cancellation.	Yes/No text; 1,118 Yes (546 Delayed, 572 Cancelled; none On Time).

10 Assumptions and Constraints

SOURCES: *Team brief (assumptions, constraints, goals); railway.csv (verified facts); DEPI Sec. 3-4 requirements. Items marked (derived) are conclusions drawn from the dataset facts.*

Assumptions

ID	Assumption
A1	The dataset covers only 4 months (journey dates 1 Jan - 30 Apr 2024, 121 days) and is the only history available for forecasting.
A2	Forecasts are produced for May 2024 only (rides, average daily revenue, ticket class demand).
A3	(derived) Price is ticket revenue: railway.csv has no cost data, so the "daily profit" goal is measured as daily revenue.
A4	(derived) The cleaned data stored in SQL Server is the single source for analysis, forecasting and Tableau.

Constraints

ID	Constraint
C1	Actual Arrival Time is empty for all 1,880 Cancelled journeys; delay cannot be calculated for them (NULL is valid).
C2	First Class is only 9.7% of tickets (3,058 of 31,653), so ticket class demand is imbalanced and First Class forecasts have less data.
C3	Reason for Delay has 8 labels for 5 causes and must be merged before any analysis; the station name 'Edinburgh' may duplicate 'Edinburgh Waverley'.
C4	(derived) 4 months is about 121 days, so Prophet can learn weekly patterns but not yearly seasonality.

11 Non-Functional Requirements

ID	Category	Requirement	How it is checked
NFR1	Performance	Each Tableau dashboard page loads in less than 3 seconds.	Measure load time of the 7 pages.
NFR2	Usability	7 focused pages, each starting with the same 4 KPI cards and sharing one date filter.	Review against the wireframes.
NFR3	Accuracy	Forecast MAPE below 15%.	MAPE on a hold-out period (Activity step "Evaluate MAPE").
NFR4	Security	Source code and documents are kept in a private GitHub repository.	Check repository visibility and members.

12 References

Ref.	Source and how it was used
S1	"Project Documentation_Final.pdf" - DEPI Project Documentation Guidelines: Section 4 (System Analysis & Design) for the required diagrams, schemas, wireframes and technology stack; timeline table for the 6 Nov 2026 date; rule that all documents are uploaded to GitHub.
S2	Team 6 Phase 2 brief: project context, goals, the 11 requirements and the page-by-page content. Its figures and data-quality statements were checked against S3 and corrected where they differ.
S3	railway.csv (31,653 rows x 18 columns) and railway_data_dictionary.csv, from the project files: every figure, example and data issue in this document was computed from them.

A Appendix A - Remaining DEPI Section 4 Items

SOURCES: *DEPI Guidelines Sec. 4: Problem Statement & Objectives; Functional and Non-Functional Requirements; Software Architecture; Component Diagram; UI/UX Guidelines. Content derived from the Team brief.*

A1. Problem Statement & Objectives

Problem. UK rail ticket records (31,653 journeys, January - April 2024) hold inconsistent labels and missing values, so decision makers have no reliable view of route revenue, punctuality, refunds and next-month demand.

- **Objective 1:** analyse profit (revenue) and routes.
- **Objective 2:** analyse delays and refunds.
- **Objective 3:** forecast the number of rides and the average daily revenue for May 2024.
- **Objective 4:** forecast ticket class demand for May 2024.

A2. Functional Requirements

ID	Requirement	Use case
FR1	Ingest railway.csv (31,653 rows, 18 columns).	Support for all
FR2	Merge the 8 Reason for Delay labels into 5 causes, validate Ticket Class and station names, and handle the empty Actual Arrival Time of Cancelled journeys.	UC2, UC5
FR3	Store the cleaned data in the SQL Server schema (Stations, Tickets, Journeys).	Support for all
FR4	Show popular routes and revenue by route, by day, by ticket type, ticket class and railcard.	UC1
FR5	Show punctuality, delay reasons and refund requests (by journey status, ticket type, ticket class and cause).	UC2, UC6
FR6	Forecast rides and average daily revenue for May 2024.	UC3, UC4
FR7	Forecast ticket class demand for May 2024.	UC5
FR8	Provide a 7-page Tableau dashboard (Overview, Routes, Revenue, Punctuality, Delays & Refunds, Customers, Forecast) with the same 4 KPI cards on every page and one date filter.	UC1 - UC6
FR9	Show customer and demand patterns: purchase channel, payment method, railcard, departure hour and booking lead time.	Supporting analysis (no new use case)

A3. Software Architecture and Component View

Architecture style: layered data pipeline (source, processing, storage, presentation). The project brief does not specify a hosting environment, so no Deployment Diagram is included.

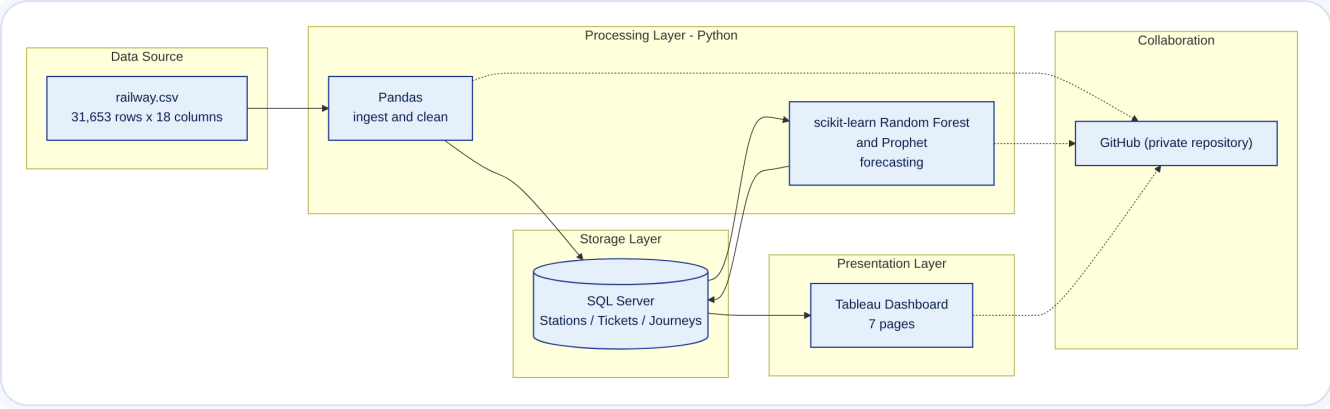


Figure 9 - *Component view: railway.csv to Pandas to SQL Server to forecasting and Tableau, with GitHub for version control*

The component view source is delivered as arch.mmd in the source bundle for GitHub.

A4. UI/UX Guidelines (proposed)

- One global date filter (January - April 2024) shared by all 7 pages; every page starts with the same 4 KPI cards, followed by 4-5 visuals.
- Consistent colours: deep blue for the main measure, orange for forecasts or delays, red for cancellations.
- Large KPI cards, readable labels and direct titles; each visual carries the business question it answers; do not rely on colour alone to separate categories.